-

Universidad de los Andes

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA

RED NEURONAL CONVOLUCIONAL PARA CLASIFICAR FENÓMENOS TRANSITORIOS ELECTROMAGNÉTICOS EN SEP

Proyecto Fin de Carrera

Autor:
Daniela Gutierrez

Asesor Principal:
Profesor Gustavo Ramos Ph.D

Julio 2024

Quiero expresar mi agradecimiento a las personas que hicieron posible la realización de este trabajo. A mis profesores mentores, Gustavo Ramos y Luis Giraldo, por su continua guía y apoyo. Un agradecimiento especial a mis compañeros y colegas, Gonzalo y Jaiverth, por sus valiosos aportes y sugerencias que enriquecieron e hicieron posible el desarrollo de este proyecto.

Agradezco también a los demás profesores del Departamento, quienes con sus enseñanzas y consejos contribuyeron a mi formación académica y al éxito de este trabajo.

A mi familia, amigos y Daniel, por su constante voz de aliento. Este logro no es solo mío, sino de todos aquellos que me han acompañado en este camino. A todos, mi más sincero agradecimiento.

I. Resumen

El estudio de la calidad de la energía en sistemas eléctricos de potencia es vital para la industria, ya que ayuda a identificar eventos perturbadores que afectan el rendimiento y la integridad de los equipos. Este proyecto desarrolla una red neuronal convolucional (CNN) para clasificar fenómenos transitorios electromagneticos generados por maniobras de energización y desenergización de bancos de condensadores y de líneas de transmisión.

Utilizando el software ATPDraw, se generaron conjuntos de datos para entrenar y validar el modelo. La metodología incluyó la automatización de simulaciones y la extracción de características relevantes. Los resultados muestran que el modelo de CNN puede clasificar eficazmente los diferentes tipos de transitorios, ofreciendo una herramienta confiable para monitorear y mejorar la calidad de la energía en sistemas eléctricos.

Palabras clave: Fenómenos transitorios, redes neuronales convolucionales, ATPDraw, Energizacion de linea de transmisión, Energizacion banco de condensadores, regresión logística

II. OBJETIVOS

Objetivo general

El objetivo general consiste en desarrollar una red neuronal convolucional que permita clasificar los fenómenos transitorios de tipo pulso y oscilatorio para tres casos: Descarga atmosférica, energización y desenergización de banco de condensadores y de línea en un sistema eléctrico de potencia con una alta confiabilidad y bajo coste computacional.

Objetivos específicos

Se han definido los siguientes objetivos específicos, encaminados a llegar a un resultado saisfactorio:

- Definir el conjunto de datos de entrenamiento, pruebas y validación, a partir de simulaciones de fenómenos transitorios electromagnéticos generadas en el software ATP Draw.
- Definir las etiquetas que se asignaran al conjunto de datos, que a su vez implicará definir que tipos de clasificaciones de transitorios abarca el modelo
- Realizar el ajuste de hiperparámetros (número de nodos, capas ocultas) de entrenamiento de la red neuronal, con el fin de alcanzar el óptimo rendimiento y precisión del modelo.
- Realizar la interpretación adecuada de los resultados, buscando entender el alcance real del modelo desarrollado y su usabilidad.

El estudio de la calidad de la potencia dentro de sistemas eléctricos de potencia es de suma importancia para la industria. Dado que, permite identificar aquellos eventos que perturban el sistema y generan operaciones fuera de las condiciones eléctricas limite, que impiden un desempeño satisfactorio de los equipos y comprometen su integridad.

Existen múltiples eventos que provocan distorsiones de onda de tensión o corriente debidas a la implementación de cargas no lineales, interacción con factores ambientales o instalación de dispositivos de potencia. Que derivan por nombrar algunos ejemplos en: desbalances de tensión, armónicos de tensión y corriente, elevaciones en la amplitud de la frecuencia, sobretensiones o sobrecorrientes transitorias.

III. FENÓMENOS TRANSITORIOS

Estos últimos, debido a los llamados fenómenos transitorios electromagnéticos, que hacen referencia a las respuestas instantáneas de un sistema y su traspaso de un estado de operación a otro, ante perturbaciones repentinas sobre la configuración inicial o la condición de estado estable de un sistema energizado con períodos de duración extremadamente cortos [1]. Es necesario recordar que este traspaso entre estados del sistema, se realiza de manera gradual a pesar de su corta duración, dadas las limitaciones de las leyes físicas de comportamiento del voltaje y corriente.

Dentro esta categoría, existen múltiples criterios de clasificación de acuerdo con su fuente de origen; externa como las descargas atmosféricas o internas por maniobras, por su forma de onda entre tipo pulso, caracterizado por una frecuencia distinta a la fundamental de frente rápido (de hasta 5 ns de tiempo de ascenso desde estado estable hasta el valor máximo del impulso) o de tipo oscilatorio, donde el valor de corriente o voltaje cambia de polaridad, que a su vez puede clasificarse entre baja, media y alta frecuencia. Para efecto de este estudio se considerara la clasificación desarrollada de acuerdo con la normativa IEEE-1159. Donde las clasificaciones del espectro de la frecuencia son como sigue: [2]

- Frecuencia de 0 Hz a 100 Hz.-Cambios de carga y fenómenos transitorios debido a la saturación
- Frecuencia de 100 Hz a 10 kHz. - Transitorios por maniobras
- Frecuencia de 10 kHz a 1 MHz.-Transitorios por descargas atmosféricas
- Frecuencia de 1 MHz a 50 MHz.-Transitorios rápidos

Para el estudio de transitorios, es necesario entonces examinar los parámetros dependientes de la frecuencia cuando se consideran modelos R,L,C. Para efecto de este estudio, se opto por la operación de un interruptor, teniendo en cuenta que la corriente y voltaje se desfasan debido al efecto capacitivo e inductivo de los diferentes elementos asociados al interruptor, lo que provoca los conocidos fenómenos transitorios por maniobras.[2]

A. *Energización de banco de condensadores*

Dentro de los sistemas de potencia, la manera de cuantificar la calidad del suministro, esta relaciona con la proximidad del modelo con las condiciones ideales, es decir voltaje y frecuencia constantes, un factor de potencia unitario y en sistemas trifásicos, que las corrientes y tensiones se encuentren balanceados. Lo anterior, puede verse afectado debido a las variaciones generadas por la carga, lo cual suele corregirse mediante compensación; que consiste en la absorción o entrega de potencia reactiva.

Existen diferentes métodos de compensación de acuerdo con las necesidades y características del sistema, sin embargo, usualmente estos consisten en la conexión ya sea en serie o paralelo de bancos de condensadores. Sin embargo, la maniobra de apertura y cierre de estos bancos, puede derivar en oscilaciones de corriente y voltaje de alta frecuencia que resultan de peligrosidad elevada y pueden generar daños sobre los equipos diseñados para la protección del sistema pero que no necesariamente tienen las características para hacer frente a corrientes capacitivas, como es el caso de interruptores automáticos.[4][5]

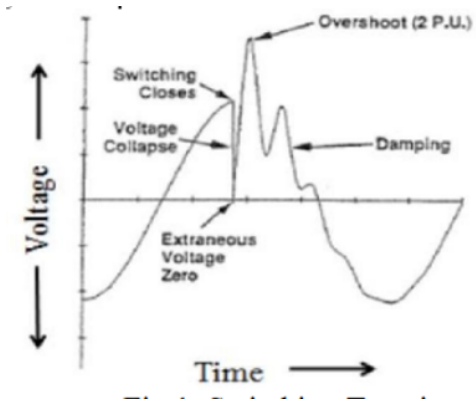

Fig. 1: Transitorio electromagnético causado por banco de condensadores [6]

La manera como se genera el transitorio está relacionada con que la activación que provoca la caída instantánea del voltaje hacia cero para intentar una rápida recuperación del voltaje. Finalmente, un voltaje fluctuante se superpone/afecta a la frecuencia del sistema, lo que puede llegar a provocar un voltaje dos veces mayor que el voltaje pico regular y una corriente de cuatro a seis veces mayor que la corriente promedio.[6] Como se observa en la Figura 1.

Existen diversas maneras como se puede originar un fenómeno transitorio de acuerdo con diferentes tipos de maniobras relacionadas con bancos de condensadores que suceden en una subestación: [6]

- Sobrecorriente de energización
- Conmutación consecutiva o en paralelo
- Transitorio de desenergización
- Amplificación de voltaje

B. Energización de línea de transmisión

En primer lugar, es necesario recordar que una línea de transmisión hace referencia a un conjunto de elementos conductores dispuestos geométricamente, de manera que permitan transportar señales eléctricas entre dos puntos.[7] Su comportamiento puede definirse como una red con parámetros distribuidos, motivo por el cual, la presencia de fenómenos transitorios puede generarse fácilmente por multitud de factores propios de la operación de la subestación. Poseen propiedades inherentes como su impedancia característica, Z_0 determinada por factores como la inductancia por longitud L, resistencia R, conductancia G y capacitancia C, la velocidad de propagación v [7] y el comportamiento que describen las señales eléctricas correspondientes a voltaje y corriente, están dadas por las llamadas ecuaciones telegráficas:

$$\frac{\partial V(z,t)}{\partial z} + L\frac{\partial I(z,t)}{\partial t} = 0$$

$$\frac{\partial I(z,t)}{\partial z} + C\frac{\partial V(z,t)}{\partial t} = 0$$

Existen distintas formas como se genera y el efecto que produce un fenómeno transitorio en una linea de transmisión, en ocasiones las ondas viajes pueden encontrarse con discontinuidades al atravesar la linea de

transmisión, ya sea por la presencia de impedancias o algún otro fenómeno, lo que da lugar a la presencia de reflexiones de onda y provoca un comportamiento específicos de fenómenos transitorios dependiendo de la configuración del sistema.[9]

Por otra parte, pueden generarse otro tipo de transitorios a causa de la conmutación de componentes en un circuito, como interruptores, relés o semiconductores de potencia deriva en picos de tensión o corriente momentáneos que pueden provocar daños permanentes en otros equipos del sistema.[8]

Dado que estos últimos, son los de interés en este trabajo, cabe señalar sus características mas generales, poseen una duración en tiempo entre los microsegundos y milisegundos, con una alta frecuencia que se ubica en el rango de kHzs a MHz con una amplitud de voltaje y corriente varias veces mayor a la nominal del sistema, como se observa en el ejemplo de una simulación de una linea cargada con -0.8 y 0.8 p.u de la Figura 2.[10]

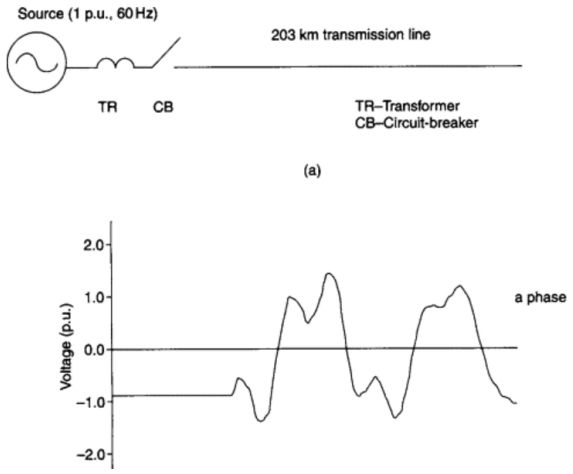

Fig. 2: Simulación simple de transitorio electromagnético [10]

A partir de las características propias de cada caso de fenómeno transitorio, se identifican las principales características comparativas en la Figura 3.

Categoría	Banco de condensadores	Línea de transmisión
Frecuencia	orden de cientos a miles de Hz	orden de decenas a cientos de Hz
Picos de voltaje	bruscos y bastante altos por carga/descarga	moderados
Transitorios	Rápidos y de corta duración	Mayor duración y más sostenidos

Fig. 3: Comparativo de características del fenómeno transitorio

El estudio calidad de potencia implica necesariamente la instalación de sistema de monitoreo, para detectar oportunamente la presencia de fenómenos que afecten los equipos y de este modo, proponer metodologías oportunas para asegurar el uso eficiente de la energía.

La propuesta de este trabajo, consiste precisamente en considerar la aplicación de redes neuronales convoluciones para el monitoreo de sistemas de potencia y la detección de fenómenos transitorios junto con la identificación de su causa de origen.

IV. REDES NEURONALES

Las redes neuronales son modelos matemáticos empleados en el campo de inteligencia artificial que simulan las interconexiones y funcionamiento de neuronas biológicas. Utilizan funciones matemáticas para modelar el comportamiento real de cada parte de la neurona. En años recientes, se han adelantado estudios para la generación de una nueva categoría en el estudio de redes neuronales para datos con estructura espacial y temporal, conocidas como CNN o redes neuronales convoluciones.

Son modelos dentro de lo que se conoce, como aprendizaje supervisado que hace referencia a introducir con sus respectivas etiquetas, diseñados de manera análoga a la estructura de la corteza visual de los mamíferos que se caracterizan por implementar operaciones de convolución entre capas. [11] Cuyo objetivo, consiste en aprender a mapear las entradas a las salidas correctas a través de la minimización de una función de perdida, siguiendo una estructura secuencial a grandes rasgos que se compone por capas: capa de entrada, capas ocultas y capa de salida, que a su vez constituidas por interconexiones de nodos conocidos como neuronas.

El proceso de aprendizaje de la red se sustenta sobre dos algoritmos fundamentales:

- **Forward propagation**

  Es el proceso a partir del cual se genera un flujo de datos desde la entrada hasta la salida (ver Fig 4). A medida que los datos se propagan por las capas y se aplican los filtros con unos pesos definidos w_i, que indican la importancia de las entradas en la salida y se añaden sesgos para que cada neurona realiza una suma ponderada de sus entradas, a la que se aplica una funciona de activación y se genera una salida. [12]

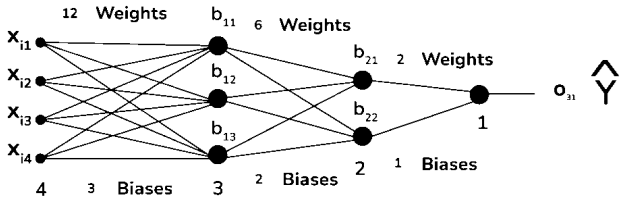

Fig. 4: forward propagation [12]

- **Back propagation**

  Es un algoritmo basado en la regla de la cadena y un método estándar en optimización conocido como descenso de gradiente, que consiste en minimizar la función de perdida en función de pesos sinápticos W_ij desplazando iterativamente a la dirección inversa del gradiente. [13]
  El entrenamiento basado en backpropagation, inicializa los pesos sinápticos aleatoriamente y modifica la dirección y actualiza el valor en la t-ésima iteración de acuerdo con la siguiente regla: [13]

$$w(t+1) = w(t) + \Delta w(t)$$

  Donde $\delta * w(t)$ es:

$$\Delta \mathbf{w}(t) = -\eta \frac{\partial J}{\partial \mathbf{w}}$$

Adicionalmente, la arquitectura de la red neuronal, consta esencialmente de 3 tipos de capas cuya cantidad y secuencialidad puede variar de acuerdo con la necesidad del modelo.

Capas convolucionales

En esta capa, se realiza una operación de convolución aplicada a una entrada, a través de un filtro o kernel, que corresponde a una matriz de números aplicada sobre un subconjunto de una imagen discriminada en píxeles normalizados, como se representa en la Figura 5.

$$(h * f)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} h(i, j) f(x - i, y - j)$$

Para generar una nueva imagen S como se muestra:

$$S(i, j) = (K * I)(i, j) = \sum_{m} \sum_{n} = I(m, n) K(i - m, j - n)$$

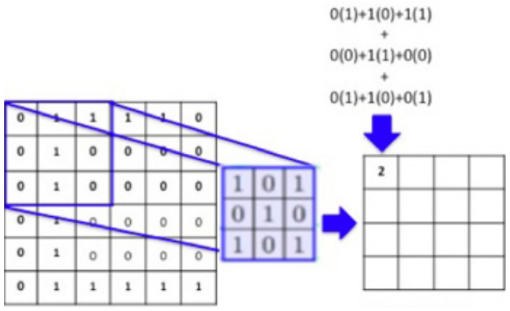

Fig. 5: representación del kernel en CNN [14]

Con el fin de extraer un patrón determinado o un único valor por una pequeña región de entrada, lo que resulta en un mapa de características o de activación. Este proceso se realiza múltiples veces de manera que se extraigan diferentes características de la imagen de entrada en diferentes regiones espaciales.

El numero de neuronas está determinado por el número de píxeles de la imagen de entrada y un canal adicional en caso de que el color sea un factor. Para efectos de este trabajo, el color resulta irrelevante. Después de realizar cada convolución se aplica también una función de (ReLu, Sigmoid, etc...) que permite introducir no linealidad en la red y de esta manera, posibilitar el análisis de patrones mas complejos.

Capas de agrupamiento

Son las capas intermedias a las capas de convolución, permite reducir las dimensiones espaciales de los mapas de activación, condensando la información que corresponde a la entrada de la siguiente capa de convolución. El objetivo consiste en reducir el número de parámetros preservando las características esenciales y evitando el sobreajuste. [15]

Suelen aplicarse operaciones de discretizacion basado en muestras, como el caso del max-pooling, que permiten reducir el numero de parámetros que se deben aprender y así reducir el coste de calculo.

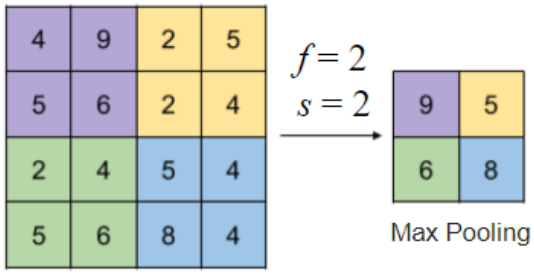

Fig. 6: ejemplo max pooling

Capas activación

Son capas diseñadas para introducir no linealidades dentro del sistema, aplicando funciones matemáticas no lineales a la entrada recibida. Cuyo objetivo, consiste en permitirle a la red interpretar patrones mas complejos. De acuerdo con las necesidades del modelo, se pueden elegir distintas funciones de activación.[14][15]

- Sigmoid $f(x) = \frac{1}{1+e^{-x}}$ se recomienda para las capas de salida de clasificación binaria, ya que puede provocar saturación que no permite la rápida minimización de la función de perdida
- ReLu $R(z) = max(0, z)$, se utiliza en las capas ocultas, favorece la convergencia del gradiente y hace mas rapido el entrenamiento
- Softmax $f(z)_j = \frac{e^{z_j}}{\sum_K^{k=1} e^{z_k}}$ Suele emplearse en las capas de salida de clasificadores multiclase
- tanh $f(x) = tanh$ Se recomienda para capas de salida de clasificacion binaria

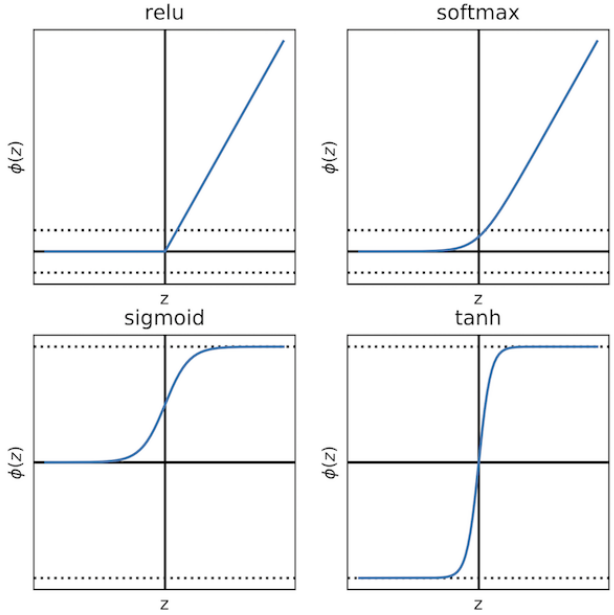

Fig. 7: funciones de activación comunes [14]

Para efecto de este trabajo, se implemento la función de activación ReLu tras una capa convolucional y se intercambio por sigmoid dado que era una clasificación binaria, sin embargo no se obtuvo un resultado diferente significativo. Se aplico en la salida la función softmax, con el objetivo de otorgar flexibilidad al modelo dado que existen múltiples casos de transitorios distintos, a pesar de que no se abordaron en el presente análisis.

En su conjunto, la red neuronal convolucional funcionaría internamente como se observa en la Figura 8.

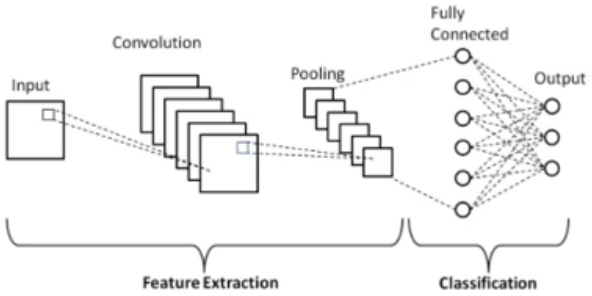

Fig. 8: Arquitectura simplificada CNN [11]

V. METODOLOGÍA

La metodología se divide en 2 etapas principales. La primera relacionada con la conformación de los datos de entrenamiento y la segunda, con el entrenamiento y ajuste de parámetros de la red neuronal. En el flujograma de la Figura 9 se observa a grandes rasgos, la metodología del proyecto.

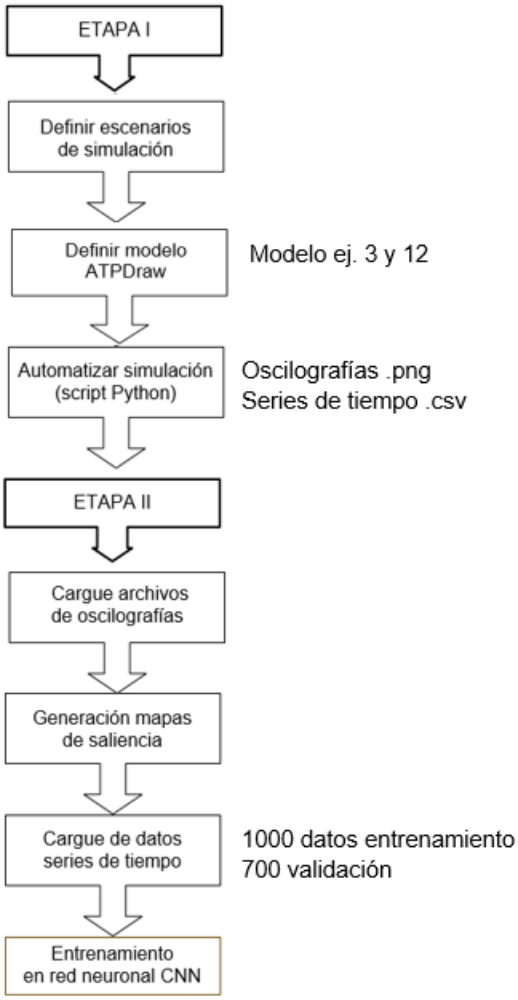

Fig. 9: Flujograma del estudio de transitorios

A. *Etapa I*

Para comenzar, se definen los escenarios generadores de fenómenos transitorios que se tendrán en cuenta para el sistema de clasificación. Se deciden utilizar 2 que suelen provocar fenómenos transitorios con mayor frecuencia en sistemas de potencia.

Adicionalmente, considerando la dificultad de acceso a bases de datos que permitan obtener con claridad datos de entrenamiento específicos para el fenómeno de estudio en particular, se opto por generar las oscilografías mediante casos de estudio simulados en el software ATPDraw, de manera que se puedan obtener datos variados y un grupo

de entrenamiento de tamaño suficiente para introducir en la red neuronal. Para el estudio, únicamente se optó por realizar variaciones sobre el tiempo de cierre de los interruptores dado que es el evento que suele generar la mayor corriente de energización.

En la Figura 10 y 11 se encuentran los casos de estudio analizados.

- Energización banco de condensadores

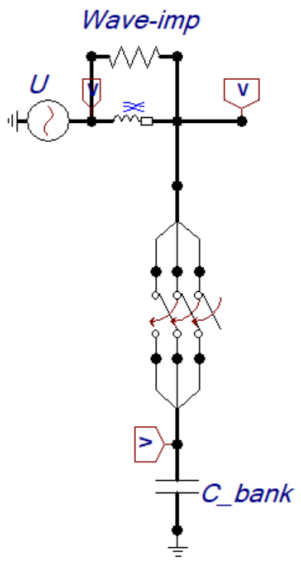

Fig. 10: Caso de estudio. Banco de condensadores

Se utiliza el caso modificado del ejemplo No. 3 disponible en ATPDraw. Se elimina la parte de linea de transmisión de modelo Pi para obtener únicamente el comportamiento del banco de condensadores. Estos son los parámetros de la simulación:

Alimentacion	$V_n(kV)$	$f(Hz)$	dT
AC	408	60	$1e^{-5}$

TABLE I: Parámetros simulación ATPDraw CASO I

- Energización de linea

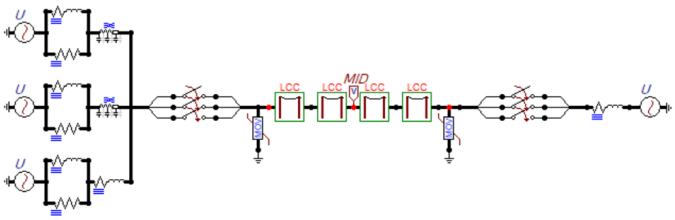

Fig. 11: Caso de estudio. Energizacion de Linea

La linea de transmisión con longitud de 50 km, modelada bajo el procedimiento Line/Cable LCC como una línea aérea, trifásica, tipo JMarti, , Rho (resistividad del suelo) = 50 Wm, Freq. init = 50 Hz. Los parámetros se encuentran en la Tabla 2.

Ph No.	$Ro(cm)$	$Z(ohm/kmDC)$	$Horiz(m)$	$Vtoweer$	$Vmid$	$Separ$
1	1.125	0.114	4.8	28.6	18.6	40
2	1.125	0.114	5	21.5	11.5	40
3	1.125	0.114	-5	21.5	11.5	40

TABLE II: Parámetros Linea de transmisión simulación ATPDraw CASO II

La carga modela con el modelo RLC3 de ATP conectada a un generador de corriente alterna (U).

$R(ohm)$	$L(mH)$	$C()$
2	100	0

TABLE III: Parametros carga conectada a generador de corriente alterna simulacion ATPDraw CASO II

Se utiliza el caso de ejemplo No. 12, un modelo disponible en ATPDraw que utiliza la técnica estadística de muestreo aleatorio de Monte Carlo. Se decide conservar los DPS, a pesar de su efecto de atenuar el fenómeno transitorio, con el objetivo de simular una instalación mas próxima a un funcionamiento real. Los valores por defecto del interruptor "Switch Statistics" del ejemplo, se utilizan como referencia, donde se indica que el T de cierre es de 35 ms y sigue una distribución gaussiana con una desviación estándar de 2 ms. Por tanto, se define en el proceso de automatizacion, que para el tiempo de cierre del interruptor se asignara un valor aleatorio del intervalo 31 a 39 ms.

VI. Automatización de simulaciones

Para obtener la cantidad de simulaciones requeridas, se realiza un proceso automatizado mediante una rutina programada en Python (Anexo 1), que permite modificar parámetros de interés (por ejemplo: impedancia, tiempo de apertura de interruptores) de los escenarios correspondiente a cada caso y asignarles un valor aleatorio directamente en el archivo .ATP, con el fin de realizar la simulación con el programa ATPDraw y graficar la oscilografía resultante mediante las librerias Scipy.io y Matplotlib.

El funcionamiento consiste en cargar un archivo de simulación base generado por ATPDraw con extensión .atp, modificar el tiempo de cierre del interruptor con un numero aleatorio dentro un rango predefinido según el caso; caso I entre [0.2500 – 0.9999s] y caso II entre [31-39ms], y guardar el archivo modificado. Posteriormente, utiliza un convertir de extensión de .atp a .pl4 y mediante el sistema de comandos corre el archivo con el simulador ATP y guarda los resultados como diccionarios, lo que puede utilizarse para graficar la oscilografía directamente o reescribir los valores de tiempo y voltaje generados en un archivo .csv como series de tiempo. El proceso descrito, se realiza la cantidad de veces definidas por el usuario y produce el resultado de cualquiera de las 3 fases definidas

aleatoriamente. En la Figura 12, se presenta un ejemplo de las oscilografias obtenidas con este proceso para ambos casos.

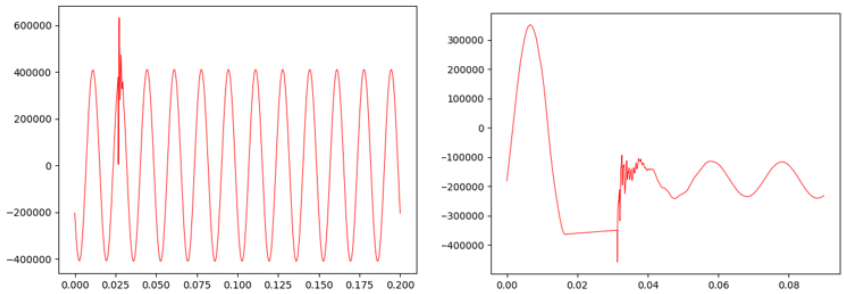

Fig. 12: Oscilografias caso I y II automatizadas en Python

Se definió el voltaje de fuente en $400000V$, un tiempo máximo $Tmax = 0.5s$ y una frecuencia $F = 20kHz$ dentro de los parámetros de cada simulación en ATPDraw, con el fin de lograr cierto grado de homogeneidad y consistencia en las simulaciones con el fin de reducir diferencias en características irrelevantes y favorecer el reconocimiento del transitorio por la red neuronal. En la Figura 13, se observa como las oscilografías de ambos casos superpuestos a partir del cargue de las series de tiempo, poseen similitud en amplitud y fase en el comportamiento en el estado estable.

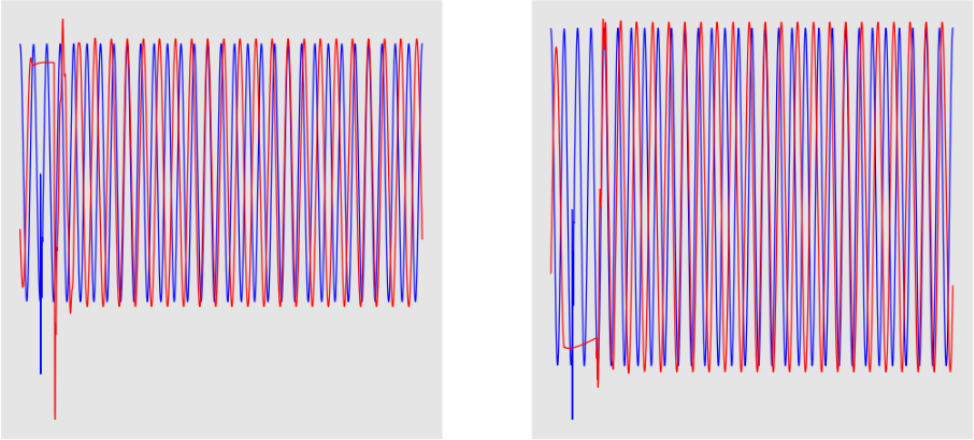

Fig. 13: Oscilografías a partir de series de tiempo superpuestas

A. Etapa II

En esta etapa se aplicaron dos aproximaciones distintas. En primer lugar, dado que las CNN están basadas en el funcionamiento de la corteza visual humana y suelen emplearse en situaciones relacionadas con el análisis de imagen, se realizaron pruebas con oscilografías producto de la simulación post-mortem a manera de imagen (Anexo 2).

En segundo lugar, se utilizaron los datos de series de tiempo correspondientes al tiempo y el voltaje por cada una de las tres fases correspondientes a cada caso de simulación. Una vez consolidad el grupo de datos de entrenamiento, que cuenta con un conjunto de 1000 resultados indistintos por fase y se reservan 700 para la validación, se establecen y asignan las etiquetas de clasificación que se espera que la red reconozca. En este caso,

ya que son dos casos únicamente, se generan 2 etiquetas "Banco" y "Linea", para clasificar transitorios de ambos tipos.

El modelo se establece principalmente con base al funcionamiento de una red convolucional (CNN), que reduce la resolución de los mapas de características utilizando una capa de max-pooling después de cada convolución, y luego los cargan en una capa ReLU para activar los mapas. Se construye en Python a través del uso de la librerías Keras y TensorFlow y se cargan los datos de entrenamiento (Anexo 3). En la Tabla 4, se especifican los parámetros de la arquitectura CNN, con 2 clases correspondientes a "Banco" y "Linea".

Parámetro	Valor
Input (Longitud datos)	8000
Canal	1
Clase	2
Tamaño Kernel	123
Filtros	16 (primera capa)
Ventana pooling	4
Activacion (convolucional)	"ReLu"
Activacion (salida)	"Softmax"

TABLE IV: Parámetros arquitectura CNN

En la Figura 14 se representa el modelo secuencial simplificado por capas de la red neuronal.

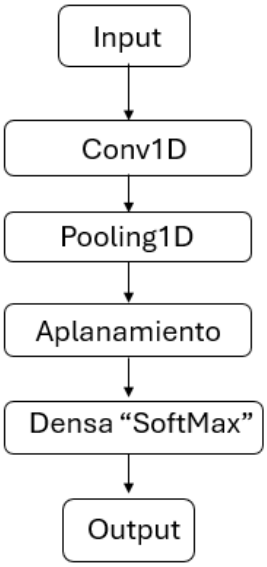

Fig. 14: Modelo por capas

El entrenamiento se realizo durante 100 epocas, con un batch size de 16 y 3 callbacks (EarlyStopping, ReduceLROnPlateau, ModelCheckpoint).

VII. Resultados

Las simulaciones realizadas arrojaron resultados que cumplen las normasIEC 60071-2, como usualmente sucedería en un sistema de potencia real.

Para aplicar efectivamente métodos de clasificación a través de redes neuronales convolucionales, es necesario tener claridad sobre la arquitectura del entrenamiento, de tal forma que sea posible diseñar una red optimizada, de allí, que se empleen métodos como los mapas de saliencia, que otorgan información estructural del modelo y proporcionan una medida cualitativa del campo de entrada como lo interpreta el modelo.

Los mapas de saliencia, hacen referencia a métodos de atribución de píxeles que destacan los píxeles mas relevantes para una determinada clasificación de imagen por una red neuronal. Es decir, expresan que tanto influye cada característica de entrada en el cambio la predicción (negativamente o positivamente). [16] En la Figura 15 y 16, se presentan los resultados obtenidos de la aplicación de esta técnica para ambos casos:

A. *Mapas de Saliencia*

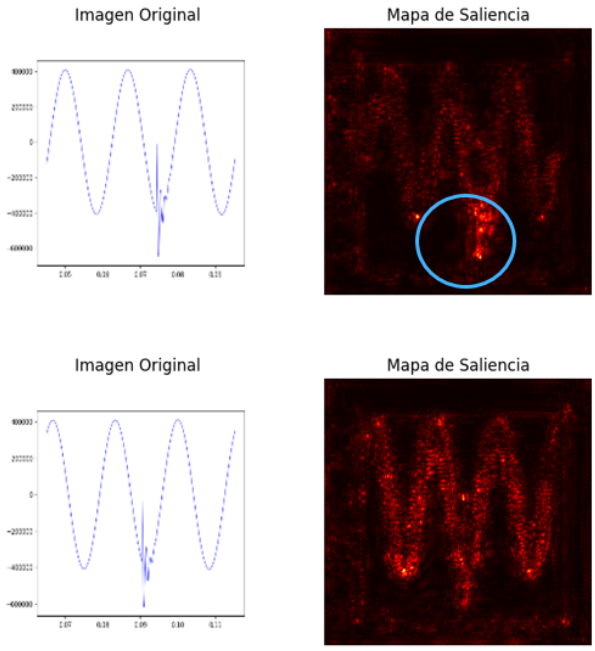

Fig. 15: Saliency maps caso I

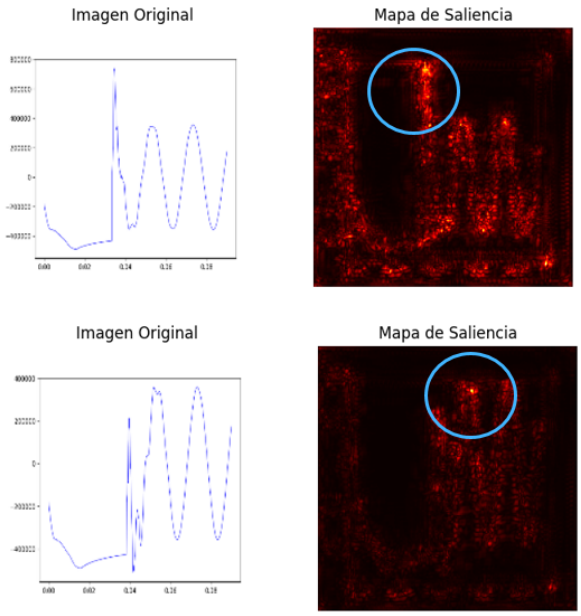

Fig. 16: Saliency maps caso II

A partir de estos resultados, se observa que el diseño de la red neuronal, prioriza el patrón repetitivo, correspondiente a la señal en estado estable para ambos casos y adicionalmente, tiene un sensibilidad a aquellas regiones de la oscilografia que presentan una picos una amplitud elevada, como se observa claramente en el caso II (Fig. 16). Lo que, en algunos escenarios se relaciona con la aparición del fenómenos transitorio, pero no esta ligada a una relación condicional necesariamente.

Para algunos escenarios correspondientes al caso I (Fig. 15), se evidencia una identificación mas precisa de la región espacial del transitorio, anteponiéndolo al comportamiento sinusoidal de estado estable de la simulación, lo cual indica que la red es capaz de identificar este fragmento de información de interés. No obstante, esto no sucede en la mayoría de los casos, por lo que existe una duda razonable sobre la viabilidad de esta arquitectura de red en la clasificación de perturbaciones causadas por fenómenos transitorios. Ya que si bien, este tipo de redes son preferibles en el análisis de patrones espaciales complejos, para efectos prácticos, la precisión temporal y la eficiencia computacional estaba mas alineada con el objetivo del trabajo y son mas próximos a las posibilidades de aplicación en la industria.

Por este motivo, se opto por aplicar la red convolucional 1D, que permite trabajar con datos secuenciales e ingresar como entrada las tablas de datos de series temporales producidos a partir de la automatización de la simulación. De este modo, se evita la perdida de resolución temporal y disminuyen los errores de conversión de datos durante el pre-procesamiento que se generan al convertir la información en imágenes. A continuación, se observan los resultados obtenidos:

B. Red convolucional 1D

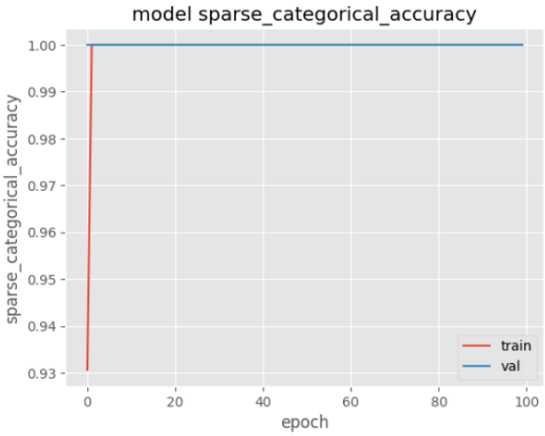

Fig. 17: Comportamiento del modelo

A través de esta métrica (Ver Fig.17), se observa como se alcanza un valor muy cercano a 1 desde las primeras épocas, con una muy reducida variación en la precisión a lo largo del entrenamiento para el comportamiento del entrenamiento. Mientras, que para el conjunto de validación, se evidencia que la red es capaz de clasificar con éxito desde las primeros épocas.

El resultado del test de precisión, que corresponde a una métrica de análisis estadístico para evaluar el rendimiento utilizado en aprendizaje automático, fue de 1.0 lo que indica que en el modelo predijo acertadamente en cada una de las instancias del conjunto de datos de prueba, es decir que cada predicción coincidió con las valores reales de las etiquetas y se presento un rendimiento perfecto.

Mientras que, el test de perdida, otra métrica que evalúa la capacidad de generalización del modelo predictivo sobre un conjunto de datos de prueba en términos de la función de pérdida definida, dio como resultado $3.314e^{-7}$ indicando que las predicciones del modelo son muy cercanas a los valores esperados y por tanto, tiene una alta precisión.

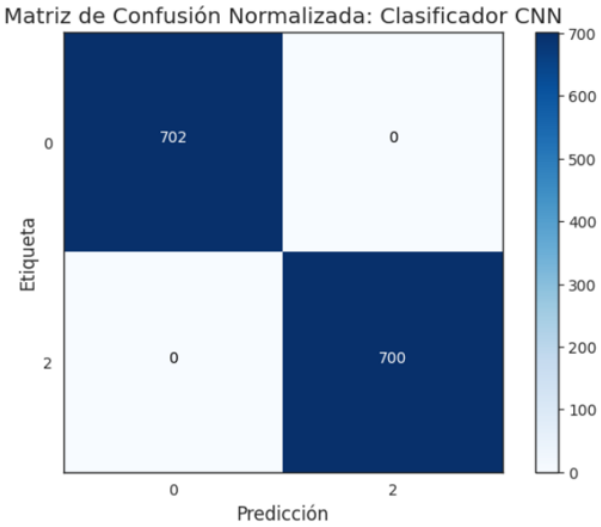

Fig. 18: Matriz de confusión

Adicionalmente, se obtuvo la matriz de confusión (ver Fig. 18); una herramienta que permite visualizar el rendimiento de un modelo de clasificación binaria. Posee 4 elementos de izquierda a derecha; Verdaderos positivos (TP) correspondiente al numero de instancias predichas correctamente para la etiqueta 1, Falsos negativos (FN), predicciones incorrectas para la etiqueta 0, Falsos positivos (FP), predicciones incorrectas para la etiqueta 1 y verdaderos negativos (TN) correspondiente al numero de instancias predichas correctamente para la etiqueta 0. Tal y como se aprecia en Figura 18, no se presento ningún falso positivo ni negativo, lo que indica un resultado ideal del comportamiento del modelo.

A pesar de los resultados favorables arrojados por el modelo, también se evidencia que se presento un ajuste extremadamente rápido y preciso a los datos, lo que indica que el modelo puede estar diseñado para abordar conjuntos de datos con mayor nivel de complejidad que los que finalmente se introdujeron. De modo que, para comprobar si es el caso, se aplico una regresión logística a los datos.

La regresión logística (RL) binaria, hace referencia a una técnica estadística que se emplea para predecir el resultado de variables categóricas en función de otras variables predictorias [17], resulta una aproximación mas sencilla a la clasificación de datos con el fin de comprobar si es posible obtener un resultado preciso con menor calculo computacional. Se replica sobre el mismo conjunto de datos y se entrena el modelo.

No obstante, se presente una reducción considerable en la precisión de las predicciones, se obteniendo un resultado de 0.76. Como puede observarse en la Figura 19, correspondiente a la matriz de confusión resultante. De modo que, se infiere que aunque el modelo CNN estaba ajustado con excesivas herramientas para el conjunto de datos ingresados, la regresión logística aun es insuficiente y requiere de ajustes al modelo para aumentar la precisión de las predicciones.

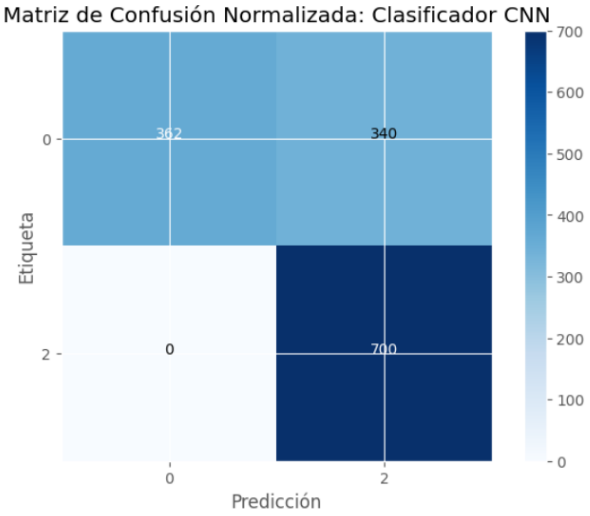

Fig. 19: Comportamiento del modelo

VIII. Conclusiones

Una aproximación con redes neuronales aplicadas a una representación visual del fenómeno, mediante las capturas de las oscilografías es limitada, en la medida en la que no garantiza que se identifique y se priorice el fenómeno de interés, en este caso, el transitorio sobre otras componentes de la simulación. Adicionalmente, el uso de imágenes implica agregar componentes visuales que no son relevantes para los datos de series de tiempo, como colores, texturas, y patrones espaciales, que pueden agregar ruido y distraer al modelo de aprender las características importantes, al igual que la posibilidad de encontrar diferentes resoluciones, puede alterar la representación de los datos provocando perdida de consistencia en los conjuntos de datos ingresados.

Los resultados obtenidos demuestran una precisión del 100% , al emplear redes convolucionales 1D sobre datos secuenciales directamente, reflejando de este modo, la viabilidad de esta estrategia como una solución efectiva a la detección oportuna de fenómenos transitorios. Aunque se aplicaron sobre datos post-mortem, son un resultado preliminar que motiva a continuar ajustando e explorando estrategias en aprendizaje supervisado con el objetivo de alcanzar el monitoreo en tiempo real y potenciar su funcionalidad.

Adicionalmente, al desarrollar una solución practica y fácil de utilizar de automatización de simulaciones a través de Python, se logro dar una respuesta a la problemática que plantea la obtención y disponibilidad de datos reales para construir el conjunto de entrenamiento para alimentar este tipo de estrategias computacionales. De tal modo, que hace posible generar datos con múltiples variaciones en sus parámetros, en poco tiempo y en un numero ilimitado de intentos, eliminando el obstáculo considerable que representaba diseñar modelos computacionales para aplicar sobre fenómenos reales en sistemas eléctricos. Para futuras aplicaciones, se recomienda realizar variaciones aleatorias en un mayor numero de parámetros en simultaneo, con el fin de otorgar mayor variabilidad a los datos y construir un conjunto de datos mas complejo y cercano a condiciones reales, de tal modo que sea posible diseñar de herramientas computacionales cada vez mas robustas y funcionales.

Entre aspectos que se podrían perfeccionar en trabajos futuros, se requiere un ajuste más exhaustivo de la arquitectura para encontrar un balance adecuado de modo que el modelo no sea demasiado complejo para los datos, evitando el sobreajuste, pero tampoco insuficiente, asegurando que tenga la capacidad de generalizar y capturar las características importantes de los fenómenos transitorios modelo para reducir el costo computacional y mejorar la eficiencia.

Adicionalmente, resulta deseable ampliar el conjunto de datos de entrenamiento, de modo que se incluyan una mayor variedad de fenómenos transitorios, bajo condiciones operativas mas cercanas a las reales y desarrollar a futuro, una herramienta robusta que pueda implementarse en la industria en tiempo real, en aras de diseñar estrategias que permitan detectar perturbaciones en los sistemas con mayor efectividad y mejoren la calidad de la potencia.

IX. Bibliografía

References

[1] UNAM (SF). *Calidad de la Energía*. UNAM. Tomado de: http://www.ptolomeo.unam.mx:8080/xmlui/bitstream/handle/132.248.52.100/293/A5.pdf?sequence=5&isAllowed=y#:~:text=En%20base%20a%20la%20magnitud,baja%2C%20media%20y%20alta%20frecuencia

[2] Universidad de Sevilla (2023).*Transitorios electromagneticos por maniobras* https://biblus.us.es/bibing/proyectos/abreproy/70611/fichero/09++C6+Transitorios+electromagn%C3%A9ticos+por+maniobras.pdf

[3] Davarpanah, M. (1971). *Analysis of factors affecting station capacitor bank switching transients*. Missouri S&T. Tomado de: https://scholarsmine.mst.edu/cgi/viewcontent.cgi?article=8238&context=masters_theses

[4] Maldonado, A. (1987). *Energización de banco de capacitores en una subestación*. Escuela Politécnica Nacional. Tomado de: https://bibdigital.epn.edu.ec/bitstream/15000/6718/1/T716.pdf

[5] Miguel, T. (2020). *Mitigation of Back-to-Back Capacitor Switching Transients on Distribution Circuits*. Puerto Rico Electric Power Authority. Tomado de: http://www.ece.uprm.edu/~lorama/TomasMiguel.pdf

[6] Gopakumar, G. (2023). *SHUNT CAPACITOR BANK SWITCHING TRANSIENTS: A TUTORIAL AND CASE STUDY*. Michigan Technological University. Tomado de: https://pages.mtu.edu/~bamork/EE5223/mip99.pdf

[7] Transmision Grupo Energia de Bogota, *¿Como se construye una linea de transmision*, 2020. Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://www.enlaza.red/content/download/26020/file/Cartilla_Li%CC%81nea_Transmisio%CC%81n_Digital_18x32.pdf.

[8] Cesar Caceres, *Metodología para la Determinación de Transitorios Electromagnéticos en la Excitación para Elementos de Líneas de Transmisión Eléctrica*, 2019. Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://laccei.org/LACCEI2019-MontegoBay/work_in_progress/WP328.pdf.

[9] W C Chew (SF).*Transients on a Transmission Line* chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://wcchew.ece.illinois.edu/chew/ece350/ee350-12.pdf

[10] Electrical Engineer's Reference Book (Sixteenth Edition), 2003.*Electromagnetic Transients* https://www.sciencedirect.com/topics/engineering/electromagnetic-transient

[11] Federico Badillo, *Redes neuronales convolucionales: un modelo de Deep Learning en imágenes diagnósticas. Revisión de tema*, 2021. Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://rcr.acronline.org/index.php/rcr/article/download/161/219/945.

[12] Mariano Rivera,(2020) *Perceptrón Multicapa en Tensorflow-Keras*, Disponible en: https://www.cimat.mx/~mrivera/cursos/aprendizaje_profundo/mlp/mlp.html

[13] Wilfrido Gomez, *Perceptrón multicapa y algoritmo backpropagation*, Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://www.tamps.cinvestav.mx/~wgomez/material/RP/MLP.pdf.

[14] V S Lakkavaram,(2019) *A review on Practical Diagnostic of Tomato Plant Diseases*, Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://www.researchgate.net/profile/Lakkavaram-Raghuveer-2/publication/332570921_A_review_on_Practical_Diagnostic_of_Tomato_Plant_Diseases/links/5cbea6aaa6fdcc1d49a872c0/A-review-on-Practical-Diagnostic-of-Tomato-Plant-Diseases.pdf?.

[15] Universidad de Zaragoza,(2019) *Introducción a las redes neuronales de convolución. Aplicación a la visión por ordenador.* Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://core.ac.uk/download/pdf/290002463.pdf

[16] Niels Morch, Kjems Ulrich,(2014) *Visualization of neural networks using saliency maps*, Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://www.researchgate.net/profile/Benny-Lautrup/publication/3623243_Visualization_of_neural_networks_using_saliency_maps/links/0fcfd50b9e2462f1f5000000/Visualization-of-neural-networks-using-saliency-maps.pdf?

[17] Biblioteca Central UNMSM,(SF) *MODELO DE REGRESIÓN LOGÍSTICA*, Disponible en: chrome-extension://gphandlahdpffmccakmbngmbjnjiiahp/https://sisbib.unmsm.edu.pe/bibvirtualdata/tesis/basic/salcedo_pc/enpdf/cap2.pdf

X. Anexos

XI. Código automatizacion de simulaciones ATPDraw

```
import os
import csv
import random
import scipy.io as sio

# Configuraciones iniciales
folder = "C:\\emtp\\atpdraw\\results\\Banco_csv"
cuantas = int(input("Ingrese número de muestras:"))
filename = "test_banco.csv"
ruta = os.path.join(folder,filename)
voltajes = []

# Proceso repetitivo para obtener diferentes vectores de voltaje
for i in range(cuantas):
    archivo_original = "C:\\emtp\\atpdraw\\results\\bancotesis.atp"
    vieja = ".03333"
    n = random.randint(2500, 9999)
    nueva = ".0" + str(n)

    archivomod = "C:\\emtp\\atpdraw\\results\\bancotesis1p.atp"
    # Modificación del archivo
    with open(archivo_original, 'r', encoding='latin-1') as f_orig, open(archivomod, 'w') a
        for linea in f_orig:
            linea_modificada = linea.replace(vieja, nueva)
            f_mod.write(linea_modificada)

    # Comandos del sistema para procesar el archivo
    os.system(r'"C:\ATP\atpmingw\tpbig.exe disk C:\emtp\atpdraw\results\bancotesis1p.atp s
    os.system(r'"C:\Pl42mat09\Pl42mat09\Pl42mat.exe C:\emtp\atpdraw\results\bancotesis1p.pl

    # Carga de datos
    res = sio.loadmat("C:\\emtp\\atpdraw\\results\\bancotesis1p.MAT")

    k = random.randint(0, 2)
    if k == 0:
        fase = 'vX0001a'
    elif k == 1:
        fase = 'vX0001b'
    elif k == 2:
        fase = 'vX0001c'

    va = res[fase].flatten()
    if i == 0:
        # Asumimos que todos los vectores de tiempo son iguales, tomamos el primero
        t = res['t'].flatten()

    voltajes.append(va)
```

```
# Escritura del CSV
with open(ruta, 'w', newline='') as csvfile:
    csvwriter = csv.writer(csvfile)
    headers = ['Tiempo'] + [f'Banco_{i+1}' for i in range(cuantas)]
    csvwriter.writerow(headers)
    # Asumimos que todos los vectores tienen la misma longitud
    for i in range(len(t)):
        fila = [t[i]] + [voltajes[j][i] for j in range(cuantas)]
        csvwriter.writerow(fila)
```

XII. Código mapas de saliencia

```
    import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications.vgg16
import VGG16, preprocess_input, decode_predictions
from google.colab import drive
import cv2
import os

# Montar Google Drive
drive.mount('/content/drive', force_remount=True)

# Cargar el modelo VGG16 preentrenado
model = VGG16(weights='imagenet')

# Directorio de imágenes
prueba_A = '/content/drive/MyDrive/Pruebas_Banco'
TAMANO = 224  # El tamaño de entrada esperado por VGG16 es 224x224

# Iterar sobre cada archivo en el directorio
for archivos in os.listdir(prueba_A):
    ruta = os.path.join(prueba_A, archivos)
    # Cargar la imagen usando OpenCV
    imagen = cv2.imread(ruta)
    if imagen is None:
        continue  # Saltar archivos que no son imágenes

    # Redimensionar la imagen al tamaño esperado
    imagen = cv2.resize(imagen, (TAMANO, TAMANO))

    # Convertir la imagen a RGB si es necesario
    if len(imagen.shape) == 2:  # La imagen es en escala de grises
        imagen = cv2.cvtColor(imagen, cv2.COLOR_GRAY2RGB)
    elif imagen.shape[2] == 4:  # La imagen tiene un canal alfa (RGBA)
        imagen = cv2.cvtColor(imagen, cv2.COLOR_BGRA2RGB)
    else:
        imagen = cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB)
```

```
    # Preprocesar la imagen para VGG16
    imagen_preprocesada = preprocess_input(np.expand_dims(imagen, axis=0))

    # Realizar predicción
    predictions = model.predict(imagen_preprocesada)
    predicted_class = np.argmax(predictions[0])

    # Calcular el mapa de saliencia
    with tf.GradientTape() as tape:
        inputs = tf.cast(imagen_preprocesada, tf.float32)
        tape.watch(inputs)
        predictions = model(inputs)
        loss = predictions[:, predicted_class]

    # Obtener el gradiente de la pérdida con respecto a la imagen de entrada
    grads = tape.gradient(loss, inputs)[0]

    # Calcular el valor absoluto del gradiente
    saliency = tf.reduce_max(tf.abs(grads), axis=-1)

    # Convertir a numpy array
    saliency = saliency.numpy().reshape((TAMANO, TAMANO))

    # Mostrar la imagen original
    plt.figure(figsize=(8, 8))
    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(imagen, cv2.COLOR_RGB2BGR))
    plt.axis('off')
    plt.title('Imagen Original')

    # Mostrar el mapa de saliencia
    plt.subplot(1, 2, 2)
    plt.imshow(saliency, cmap='hot')
    plt.axis('off')
    plt.title('Mapa de Saliencia')

    plt.show()
```

XIII. Código red neuronal CNN

```
import math                                     # Operaciones matematicas
import numpy as np                              # Vectores y matrices
import pandas as pd                             # Tablas y manipulación de datos
import matplotlib.pyplot as plt                 # Grafícas
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import warnings                                 # 'No molestar' modo
```

```
warnings.filterwarnings('ignore')

plt.style.use('ggplot')

def read_data(filename):
    import pandas as pd

    dataset = np.genfromtxt(filename, delimiter=',')
    dataframe = pd.read_csv(filename)
    labels = dataframe.columns.tolist()
    new_labels = []
    for label in labels:
        if "linea" in label.lower():
            new_labels.append("Linea")
        else:
            new_labels.append("Banco")

    new_labels = np.array(new_labels)
    time = dataset[:, 0]
    data = dataset[1:, :]
    data = data.T

    return  data, new_labels,time

banco_entreno = 'prueba_banco.csv'
banco_test = 'test_banco.csv'
x1_train = read_data(banco_entreno)[0]
y1_train = read_data(banco_entreno)[1]
x1_test = read_data(banco_test)[0]
y1_test = read_data(banco_test)[1]

linea_entreno = 'prueba_linea.csv'
linea_test = 'test_linea.csv'
x2_train = read_data(linea_entreno)[0]
y2_train = read_data(linea_entreno)[1]
x2_test = read_data(linea_test)[0]
y2_test = read_data(linea_test)[1]

x_train = np.concatenate((x1_train, x2_train), axis=0)
y_train = np.concatenate((y1_train, y2_train), axis=0)

x_test = np.concatenate((x1_test, x2_test), axis=0)
y_test = np.concatenate((y1_test, y2_test), axis=0)


import sklearn
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score,
mean_squared_error, confusion_matrix, classification_report
```

```
from sklearn import preprocessing

def scaler(x):
    scaler = sklearn.preprocessing.MinMaxScaler(feature_range=(0,1))
    scaler.fit(np.transpose(x))
    x = np.transpose(scaler.transform(np.transpose(x)))
    return x

tiempo_b = x1_train.shape[1]
tiempo_l = x2_train.shape[1]

seed = 128
rng = np.random.RandomState(seed)

def plot_waveform():
    fig, axes = plt.subplots(nrows=3,ncols=3 , figsize = (20,20))
    for i, ax in enumerate(axes.flat):
        ax.plot(range(tiempo_b), x1_train[i, :], color='b',linewidth=0.9)
        ax.plot(range(tiempo_l), x2_train[i, :], color='r',linewidth=0.9)
        print()
        ax.set_xticks([])
        ax.set_yticks([])
    plt.show()

plot_waveform()

import tensorflow as tf
from tensorflow import keras

x_train = x_train.reshape((x_train.shape[0], x_train.shape[1], 1))
x_test = x_test.reshape((x_test.shape[0], x_test.shape[1], 1))

input = (10001,1)
clase = 2 # Numero de la clase que se debe cambiar para tipo de evento
filtros = 16 # Numero de Filtros
kernel = 123 # Tamaño del Kernel
padding = 'same'
pooling_size = 4
dense_activation = 'softmax'
conv_activation = 'sigmoid'

def make_model(input_shape, num_classes, num_filters, kernel_size, padding,
conv_activation, pooling_size, dense_activation):
     model = keras.Sequential([
       keras.layers.Input(shape=input_shape),
       keras.layers.Conv1D(filters=num_filters, kernel_size=kernel_size,
       padding=padding, activation=conv_activation),
       keras.layers.MaxPooling1D(pool_size=pooling_size),
       keras.layers.Flatten(),
```

```
        keras.layers.Dense(num_classes, activation=dense_activation)])
     return model

model = make_model(
    input_shape=input,
    num_classes= clase,
    num_filters= filtros,
    kernel_size= kernel,
    padding='same',
    conv_activation='sigmoid',
    pooling_size=4,
    dense_activation='softmax'
)

from sklearn.preprocessing import LabelEncoder

# Crear un objeto LabelEncoder
encoder = LabelEncoder()

# Ajustar el encoder a las etiquetas de entrenamiento y transformarlas
y_train = encoder.fit_transform(y_train)
y_test = encoder.fit_transform(y_test)

epochs = 100
batch_size = 16
callbacks = [
    keras.callbacks.ModelCheckpoint(
        "best_model.h5", save_best_only=True, monitor="val_loss"
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=20, min_lr=0.0001
    ),
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=50, verbose=1),
]
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["sparse_categorical_accuracy"],
)
history = model.fit(
    x_train,
    y_train,
    batch_size=batch_size,
    epochs=epochs,
    callbacks=callbacks,
    validation_split=0.2,
    verbose=1,
)
```

```
model = keras.models.load_model("best_model.h5")

test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test accuracy", test_acc)
print("Test loss", test_loss)

metric = "sparse_categorical_accuracy"
plt.figure()
plt.plot(history.history[metric])
plt.plot(history.history["val_" + metric])
plt.title("model " + metric)
plt.ylabel(metric, fontsize="large")
plt.xlabel("epoch", fontsize="large")
plt.legend(["train", "val"], loc="best")
plt.show()
plt.close()

def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    print(cm)

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('Etiqueta')
    plt.xlabel('Predicción')

import seaborn as sns
from collections import Counter
import itertools
```

```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis = 1)

sns.set_style('white')
class_names = ['0',clase]
plot_confusion_matrix(confusion_matrix(y_test,y_pred_classes),
                      classes= class_names,
                      title='Matriz de Confusión Normalizada: Clasificador CNN')

print(classification_report(y_test, y_pred_classes))
```